

DESARROLLO DE INTERFACES

Las 10 Heurísticas De Nielsen



1. Análisis de las 10 Heurísticas de Usabilidad de Jakob Nielsen.....	3
1. Visibilidad del estado del sistema.....	3
2. Coincidencia entre el sistema y el mundo real.....	4
3. Control y libertad del usuario.....	6
4. Consistencia y estándares.....	7
5. Prevención de errores.....	8
6. Reconocimiento antes que recuerdo.....	10
7. Flexibilidad y eficiencia de uso.....	11
8. Diseño estético y minimalista.....	12
9. Ayuda a reconocer, diagnosticar y recuperarse de errores.....	13
10. Ayuda y documentación.....	14
2. Principios de Accesibilidad (WCAG) aplicados al proyecto.....	15
Perceptible.....	16
Operable.....	17
Comprensible.....	18
Robusta.....	19

1. Análisis de las 10 Heurísticas de Usabilidad de Jakob Nielsen

1. Visibilidad del estado del sistema

Definición:

El sistema debe mantener al usuario siempre informado sobre lo que está sucediendo, mediante una retroalimentación adecuada y en un tiempo razonable. El usuario no debería preguntarse “¿ha pasado algo?” después de pulsar un botón.

Ejemplos generales:

- Un mensaje "Archivo guardado correctamente" al guardar un documento.
- Un botón que se desactiva temporalmente tras hacer clic para indicar que se ha recibido la acción.

Aplicación en SummerHeat:

En SummerHeat, se informa claramente al usuario del resultado de sus acciones mediante mensajes emergentes (JOptionPane) en operaciones clave como:

- Guardar registro
- Imprimir a documento
- Crear nuevo registro

También la cabecera "Gestión Apartamentos Turísticos - SummerHeat" está siempre visible en la ventana principal y en el diálogo de alta, indicando el contexto de la aplicación.

Prueba en el proyecto:

En VentanaAltaPiso.java, al pulsar el botón Guardar:

```

// LISTENER DEL BOTÓN GUARDAR
// por defecto
getRootPane().setDefaultButton(bGuardar);
bGuardar.addActionListener(e -> {
    if (validarFormularios()) {
        JOptionPane.showMessageDialog(this, "Registro Guardado", "Registro Guardado",
            JOptionPane.INFORMATION_MESSAGE);
    }
});

```

Y al pulsar Imprimir:

```

// LISTENER DEL BOTON IMPRIMIR
bImprimir.addActionListener(e -> {
    if (validarFormularios()) {
        // Rellenar el Resumen
        resumen.setNombre(datosArrendado.txtNombre.getText());
        resumen.setApellido(datosArrendado.txtApellido.getText());
        resumen.setDni(datosArrendado.txtDni.getText());
        resumen.setTelefono(datosArrendado.txtTelefono.getText());

        resumen.setDireccion(datosInmueble.txtDireccion.getText());
        resumen.setProvincia((String) datosInmueble.cbProvincia.getSelectedItem());
        resumen.setNumHuespedes((int) datosInmueble.spNumHuespedes.getValue());
        resumen.setNumCamas((int) datosInmueble.spNumCamas.getValue());
        resumen.setTipoCama((String) datosInmueble.cbTipoCamas.getSelectedItem());
        resumen.setNumBanos((int) datosInmueble.spNumBanos.getValue());
        resumen.setPrecio(datosInmueble.txtPrecioMinimo.getText());

        JOptionPane.showMessageDialog(this, "Registro Imprimido", "Correcto", JOptionPane.INFORMATION_MESSAGE);
    }
});

```

2. Coincidencia entre el sistema y el mundo real

Definición:

El sistema debe hablar el lenguaje del usuario, usando palabras, frases y conceptos que le resulten naturales. Debe seguir las convenciones del mundo real, en lugar de usar términos internos o técnicos.

Ejemplos generales:

- Usar "Papelera" en vez de "Directorio de eliminados".
- Mostrar un icono de calendario para seleccionar fechas.
- Usar "Guardar" en vez de "Persistir datos en almacenamiento".

Aplicación en SummerHeat:

Toda la terminología usada está alineada con el mundo del alquiler turístico y del usuario final:

- “Dirección”, “Provincia”, “Número de huéspedes”, “Número de baños”, “Precio mínimo/día”.
- Botones: “Alta Piso”, “Baja Piso”, “Nuevo”, “Guardar”, “Imprimir a Documento”.
- Campos como DNI y teléfono, habituales en trámites reales.

Prueba en el proyecto:

En DatosInmueble.java:

```
public DatosInmueble() {  
    setBackground(COLOR_FONDO_PANELES);  
  
    // ETIQUETAS  
    lDireccion = new JLabel("Direccion: ");  
    lProvincia = new JLabel("Provincia: ");  
    lFechaAlta = new JLabel("Fecha de alta: ");  
    lFechaFinal = new JLabel("Fecha final disponibilidad: ");  
    lNumHuespedes = new JLabel("Numero de huespedes: ");  
    lNumDormitorios = new JLabel("Numero de dormitorios: ");  
    lNumBanos = new JLabel("Numero de baños: ");  
    lNumCamas = new JLabel("Numero de camas: ");  
    lTipoCamas = new JLabel("Tipo de camas: ");  
    lNinos = new JLabel("¿Niños?: ");  
    lExtrasNinos = new JLabel("Extras Niños: ");  
    lEdadNinos = new JLabel("Edad: ");  
    lImagenes = new JLabel("Imagenes: ");  
    lPrecioMinimo = new JLabel("Precio Minimo (precio minimo/dia): ");
```

En Ventana.java (menú):

```
// -----  
// MENU  
// -----  
miBarra = new JMenuBar();  
  
archivo = new JMenu("Archivo");  
registro = new JMenu("Registro");  
ayuda = new JMenu("Ayuda");  
  
salir = new JMenuItem("Salir");  
altaPiso = new JMenuItem("Alta Piso");  
bajaPiso = new JMenuItem("Baja Piso");  
acercaDe = new JMenuItem("Acerca De");
```

3. Control y libertad del usuario

Definición:

Los usuarios suelen hacer acciones por error. El sistema debe ofrecer “salidas de emergencia” claras para deshacer acciones o cancelar procesos sin complicaciones.

Ejemplos generales:

- Botón “Cancelar” al editar un formulario.
- Combinación Ctrl+Z para deshacer.
- Opción “Salir sin guardar” en un editor de texto.

Aplicación en SummerHeat:

- Se incluye un botón “Nuevo” que limpia todo el formulario y restaura valores por defecto.
- El menú Archivo → Salir confirma con un diálogo si realmente se quiere cerrar la aplicación.
- En el checkbox de “Niños”, al desmarcar, se oculta el panel y se resetean los datos asociados.

Prueba en el proyecto:

Botón Nuevo en VentanaAltaPiso.java:

```
//Boton nuevo
bNuevo.addActionListener(e -> {

    // Rellenar el Resumen
    resumen.setNombre("-");
    resumen.setApellido("-");
    resumen.setDni("-");
    resumen.setTelefono("-");

    resumen.setDireccion("-");
    resumen.setProvincia("-");
    resumen.setNumHuespedes(1);
    resumen.setNumCamas(1);
    resumen.setTipoCama("-");
    resumen.setNumBanos(1);
    resumen.setPrecio(datosInmueble.txtPrecioMinimo.getText());

    // Datos Arrendador en blanco:
    datosArrendado.txtNombre.setText("");
    datosArrendado.txtApellido.setText("");
    datosArrendado.txtDni.setText("");
    datosArrendado.txtTelefono.setText("");

    // Datos Inmueble en blanco:

    // JTextField
    datosInmueble.txtDireccion.setText("");

    // JSpinner
    datosInmueble.spNumHuespedes.setValue(1);
    datosInmueble.spNumDormitorios.setValue(1);
    datosInmueble.spNumBanos.setValue(1);
    datosInmueble.spNumCamas.setValue(1);
}
```

```

// check() que devuelve que si devuelve o sea activo
if (datosInmueble.chkNinos.isSelected()) { // si esta seleccionado o sea activo
    datosInmueble.chkNinos.doClick(); // lo ponemos a false y si no esta activo no lo tocamos
}
// JComboBox
datosInmueble.cbProvincia.setSelectedItem("Álava");
datosInmueble.cbTipoCamas.setSelectedItem("Cama Simple");

// JDateChooser
datosInmueble.miCalendar.setTime(datosInmueble.hoy);
datosInmueble.miCalendar.add(Calendar.YEAR, 1);
datosInmueble.dcFechaFinal.setMinSelectableDate(datosInmueble.miCalendar.getTime());
datosInmueble.dcFechaFinal.setDate(datosInmueble.dcFechaFinal.getMinSelectableDate());
datosInmueble.dcFechaAlta.setDate(datosInmueble.hoy);

JOptionPane.showMessageDialog(this, "Registro Nuevo", "Nuevo Registro", JOptionPane.INFORMATION_MESSAGE);
});

```

Confirmación de salida en Ventana.java:

```

salir.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        // TODO Auto-generated method stub
        int respuesta = JOptionPane.showConfirmDialog(null, "¿Esta seguro de Salir?", "Salir",
            JOptionPane.YES_NO_OPTION, JOptionPane.QUESTION_MESSAGE);

        if (respuesta == JOptionPane.YES_OPTION) {
            System.exit(0);
        }
    }
});

```

4. Consistencia y estándares

Definición:

Los usuarios no deberían tener que preguntarse si palabras, acciones o elementos significan lo mismo. El sistema debe ser consistente consigo mismo y seguir los estándares de la plataforma.

Ejemplos generales:

- Botones "OK" y "Cancelar" siempre en el mismo orden.
- Mismo estilo de iconos y colores en todas las pantallas.
- Misma palabra ("Guardar") para la misma acción en todo el sistema.

Aplicación en SummerHeat:

- Se utiliza siempre el mismo esquema de colores (azul acento, gris de fondo).
- Los botones de acciones críticas tienen estilo homogéneo y texto consistente ("Guardar", "Nuevo", "Imprimir a Documento").
- Se usa el mismo tipo de componente para las mismas cosas: JSpinner para números, JComboBox para listas, JDateChooser para fechas.

Prueba en el proyecto:

Consistencia en botones del panel sur:

```
// BOTONES
JButton bImprimir = new JButton("Imprimir a Documento", iconoImprimir);
JButton bNuevo = new JButton("Nuevo", iconoNuevo);
JButton bGuardar = new JButton("Guardar", iconoGuardar);

JPanel panelBotonesResumen = new JPanel(new FlowLayout(FlowLayout.CENTER, 15, 5));
panelBotonesResumen.add(bImprimir);
panelBotonesResumen.add(bNuevo);
panelBotonesResumen.add(bGuardar);
```

Mismos colores declarados como constantes:

```
public class VentanaAltaPiso extends JDialog {

    // --- PALETA DE COLORES ---
    private static final Color COLOR_FONDO_PANELES = Color.decode("#DEE0E0");
    private static final Color COLOR_TEXTO_PRINCIPAL = Color.decode("#2C3E50");
    private static final Color COLOR_ACENTO = Color.decode("#2980B9");
```

5. Prevención de errores

Definición:

Es mejor evitar que un problema ocurra que mostrar un mensaje después. El sistema debe diseñarse para minimizar la probabilidad de errores del usuario.

Ejemplos generales:

- Desactivar el botón “Guardar” hasta que los campos obligatorios estén rellenos.
- Limitar un campo numérico con un Spinner en vez de permitir texto libre.
- Validar un email en tiempo real mientras se escribe.

Aplicación en SummerHeat:

- Uso de JSpinner con rangos definidos para evitar números fuera de límites (huéspedes, baños, camas).
- InputVerifier en DNI, teléfono y dirección para evitar formatos incorrectos antes de permitir cambiar el foco.
- Fechas con minSelectableDate para impedir seleccionar fechas pasadas o inconsistentes.

Prueba en el proyecto:

Ejemplo de JSpinner:

```
// Numero de huespedes (JSpinner)
splNumHuespedes = new JSpinner(new SpinnerNumberModel(1, 1, 8, 1)); // 1 valor inicial, 2 valor minimo, 3 valor
                                                                    // maximo, 4 incremento [por ejemplo de 1 en
                                                                    // 1, de 2 en 2 etc]
```

InputVerifier en DNI:

```
98 // Dni
99 txtDni = new JTextField(16);
100 // Ayuda contextual
101 txtDni.setToolTipText("Introduce un DNI valido, 8 Digitos y 1 Letra (MAYUSCULA)");
102 // verificamos 8 Digitos y 1 Letra
103 txtDni.setInputVerifier(new InputVerifier() {
104     @Override
105     public boolean verify(JComponent input) {
106         JTextField dni = (JTextField) input;
107         String texto = dni.getText().trim(); // Le quitamos los espacios
108         // Solo permitir salir si son 8 digitos y 1 Letra exactos
109         if (texto.matches("\\d{8}[A-Z]")) {
110             // Restauramos el borde:
111             dni.setBorder(UIManager.getBorder("TextField.border"));
112             return true; // puedes salir
113         } else {
114             // Podriamos mostrar un mensaje
115             JOptionPane.showMessageDialog(
116                 tf, "El DNI debe tener exactamente 8 digitos y 1 letra.",
117                 "Error de validación", JOptionPane.WARNING_MESSAGE
118             );
119             // Borde en Rojo
120             dni.setBorder(BorderFactory.createLineBorder(Color.RED));
121             return false; // No puede salir hasta corregir
122         }
123     }
124 });
```

6. Reconocimiento antes que recuerdo

Definición:

Los usuarios no deberían tener que recordar información de una parte del sistema a otra. Las opciones y acciones deben ser visibles o fácilmente recuperables.

Ejemplos generales:

- Menús visibles con todas las opciones principales.
- Placeholders en campos de formulario indicando el formato.
- Listas desplegables con valores disponibles.

Aplicación en SummerHeat:

- Uso de JComboBox para provincias y tipos de cama, el usuario no tiene que recordar nombres exactos.
- Tooltips explicando formatos, por ejemplo en DNI o teléfono.
- Pestañas de resumen donde se puede ver todo lo que se ha introducido sin recordar datos.

Prueba en el proyecto:

Tooltip en DNI:

text

```
// Ayuda contextual  
txtDni.setToolTipText("Introduce un DNI valido, 8 Digitos y 1 Letra (MAYUSCULA)");  
// verificamos 8 Digitos y 1 Letra
```

Combo de provincias:

```
// Provincia (ComboBox)  
String[] provincias = { "Álava", "Albacete", "Alicante", "Almería", "Asturias", "Ávila", "Badajoz", "Barcelona",  
    "Burgos", "Cáceres", "Cádiz", "Cantabria", "Castellón", "Ceuta", "Ciudad Real", "Córdoba", "La Coruña",  
    "Cuenca", "Gerona", "Granada", "Guadalajara", "Guipúzcoa", "Huelva", "Huesca", "Jaén", "León", "Lérida",  
    "Logroño", "Lugo", "Madrid", "Málaga", "Murcia", "Navarra", "Orense", "Palencia", "Palmas (Las)",  
    "Pontevedra", "La Rioja", "Salamanca", "Tarragona", "Santa Cruz de Tenerife", "Teruel", "Toledo",  
    "Valencia", "Valladolid", "Vizcaya", "Zamora", "Zaragoza" };  
cbProvincia = new JComboBox<>(provincias);  
cbProvincia.setPreferredSize(new Dimension(200, 21));
```

Resumen en pestañas:

```
// Añadir pestañas  
this.addTab("Arrendador", pArrendador);  
this.addTab("Inmueble", pInmueble);
```

7. Flexibilidad y eficiencia de uso

Definición:

El sistema debe permitir que tanto usuarios novatos como expertos sean eficientes. Los expertos pueden usar atajos, mientras que los novatos requieren interfaces más guiadas.

Ejemplos generales:

- Atajos de teclado (Ctrl+S, Ctrl+Z).
- Menús contextuales para usuarios avanzados.
- Autofill de campos repetitivos.

Aplicación en SummerHeat:

- Atajos de teclado en el menú principal (Ctrl+A para Alta Piso, Ctrl+B para Baja, etc.).
- Botón por defecto para que pulsar Enter ejecute guardar o alta.
- Cálculo automático del precio mínimo sin que el usuario tenga que escribirlo manualmente.

Prueba en el proyecto:

Atajos en Ventana.java:

```
this.setJMenuBar(miBarra);  
  
archivo.setMnemonic(KeyEvent.VK_A); // Alt + A  
registro.setMnemonic(KeyEvent.VK_R); // Alt + R  
ayuda.setMnemonic(KeyEvent.VK_Y); // Alt + Y  
  
salir.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_F4, KeyEvent.CTRL_DOWN_MASK)); // Control + F4  
altaPiso.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_A, KeyEvent.CTRL_DOWN_MASK)); // Control + A  
bajaPiso.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_B, KeyEvent.CTRL_DOWN_MASK)); // Control + B  
acercaDe.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_D, KeyEvent.CTRL_DOWN_MASK)); // Control + D
```

Botón por defecto:

```
// Boton por defecto de la ventana - al darle a Enter se abre sol:  
getRootPane().setDefaultButton(bAnadir);
```

Cálculo automático del precio:

```
// Calcular Precio, lo llamamos en los 3 casos que tenemos:  
cbTipoCamas.addActionListener(e -> calcularPrecio());  
spNumBanos.addChangeListener(e -> calcularPrecio());  
chkNinos.addActionListener(e -> calcularPrecio());  
spNumCamas.addChangeListener(e -> calcularPrecio());  
  
// para ponerle los valores por defecto llamare al metodo si no, no me muestra  
// nada:  
calcularPrecio();
```

8. Diseño estético y minimalista

Definición:

El sistema no debe mostrar información irrelevante o innecesaria. Cada unidad extra de información compite con las relevantes y reduce su visibilidad.

Ejemplos generales:

- Formularios sin texto redundante ni adornos excesivos.
- Uso de una paleta de colores limitada y coherente.
- Evitar recargar la pantalla con demasiados botones.

Aplicación en SummerHeat:

- Se ha unificado la paleta de colores (gris claro de fondo, azul acento, texto oscuro).
- Los paneles “Datos Arrendador” y “Datos Inmueble” tienen bordes con título y contenido ordenado.
- Se han ocultado elementos no requeridos (por ejemplo, el panel de niños solo se muestra si se selecciona el checkbox).

Prueba en el proyecto:

Paleta definida:

```
// --- PALETA DE COLORES ---
private static final Color COLOR_FONDO_PANELES = Color.decode("#DEE0E0");
private static final Color COLOR_TEXTO_PRINCIPAL = Color.decode("#2C3E50");
private static final Color COLOR_ACENTO = Color.decode("#2980B9");
```

Uso de TitledBorder:

```
// titulo en el borde, queda regulin:
setBorder(BorderFactory.createTitledBorder("Datos Arrendador"));

//Queda mucho mejor (lo busque):
setBorder(
    BorderFactory.createTitledBorder(
        BorderFactory.createEtchedBorder(),
        "Datos Arrendador",
        TitledBorder.CENTER,
        TitledBorder.TOP
    )
);
```

9. Ayuda a reconocer, diagnosticar y recuperarse de errores

Definición:

Los mensajes de error deben expresarse en lenguaje claro (sin códigos), indicar con precisión el problema y sugerir una solución.

Ejemplos generales:

- "La contraseña debe tener al menos 8 caracteres" en lugar de "Error 102".
- Resaltar el campo incorrecto.
- Mostrar sugerencias para corregir el error.

Aplicación en SummerHeat:

Cuando hay errores en el formulario (nombre vacío, DNI incorrecto, teléfono incorrecto, dirección vacía), se muestra un mensaje claro con:

- Una explicación (por ejemplo: "El teléfono debe tener exactamente 9 dígitos").
- Un foco automático en el campo que está mal.

Prueba en el proyecto:

En validarFormularios():

```
String dni = datosArrendado.txtDni.getText().trim();
if (dni.isEmpty() || !dni.matches("\\d{8}[A-Z]")) {
    JOptionPane.showMessageDialog(this, "El DNI debe tener exactamente 8 dígitos y 1 letra MAYUSCULA.",
        "Error: DNI invalido", JOptionPane.ERROR_MESSAGE);
    datosArrendado.txtDni.requestFocus();
    return false;
}
```

10. Ayuda y documentación

Definición:

Aunque lo ideal es que el sistema se pueda usar sin documentación, a veces es necesaria cierta ayuda. Debe ser fácil de buscar, centrada en la tarea e integrada con el sistema.

Ejemplos generales:

- Tooltips al pasar el ratón sobre un icono.
- Sección "Acerca de" con información de la aplicación.
- Mensajes de ayuda contextual.

Aplicación en SummerHeat:

- Tooltips en campos clave (DNI, teléfono, dirección, extras niños).
- Menú "Ayuda → Acerca De" con la información de la aplicación, versión y autor.
- Los bordes titulados ayudan a entender qué agrupa cada panel ("Datos Arrendador", "Datos Inmueble").

Prueba en el proyecto:

Tooltip:

```
txtTelefono = new JTextField(16);  
// Ayuda contextual  
txtTelefono.setToolTipText("Introduce un numero de telefono 9 Digitos");  
// Ayuda contextual
```

Acerca de:

text

```
acercaDe.addActionListener(new ActionListener() {  
    @Override  
    public void actionPerformed(ActionEvent e) {  
        // TODO Auto-generated method stub  
        JOptionPane.showMessageDialog(null,  
            "Aplicación: Gestión Apartamentos Turísticos - SummerHeat\n" + "Empresa: SummerHeat\n"  
            + "Version: 1.0\n" + "Autor: Ousama Kassimi\n" + "Fecha Creacion: Noviembre 2025",  
            "Acerca De", JOptionPane.INFORMATION_MESSAGE);  
    }  
});
```

2. Principios de Accesibilidad (WCAG) aplicados al proyecto

Aquí usamos los 4 grandes principios WCAG:

1. Perceptible
2. Operable
3. Comprensible
4. Robusta

Perceptible

Definición:

La información y los componentes de la interfaz deben presentarse de manera que los usuarios puedan percibirlos. Nada debe ser invisible para todos sus sentidos.

Ejemplos generales:

- Contraste suficiente entre texto y fondo.
- Texto que no depende solo del color para transmitir información.
- Tamaño de fuente legible.

Aplicación en SummerHeat:

- Se ha elegido una paleta con fondo gris claro (#DEE0E0), texto azul oscuro (#2C3E50) y acento azul (#2980B9) con alto contraste.
- Los campos con error no solo se marcan en rojo, también muestran mensajes claros por diálogo.
- Los títulos de panel están en bordes (visual y textual).

Prueba en el proyecto:

Colores:

```
// --- PALETA DE COLORES ---  
private static final Color COLOR_FONDO_PANELES = Color.decode("#DEE0E0");  
private static final Color COLOR_TEXTO_PRINCIPAL = Color.decode("#2C3E50");  
private static final Color COLOR_ACENTO = Color.decode("#2980B9");
```

Bordes y títulos:

```
// titulo en el borde, queda regulin:  
// setBorder(BorderFactory.createTitledBorder("Datos Arrendador"));  
  
//Queda mucho mejor (lo busque):  
setBorder(  
    BorderFactory.createTitledBorder(  
        BorderFactory.createEtchedBorder(),  
        "Datos Arrendador",  
        TitledBorder.CENTER,  
        TitledBorder.TOP  
    )  
);
```

Operable

Definición:

Los componentes de la interfaz deben ser utilizables. Esto incluye ser manejables por teclado, dar tiempo suficiente para realizar tareas y evitar patrones que causen problemas (como parpadeos excesivos).

Ejemplos generales:

- Navegación por teclado (Tab, Enter, Esc).
- Atajos de teclado.
- Botones suficientemente grandes y separados.

Aplicación en SummerHeat:

- Todos los componentes Swing son accesibles con teclado (Tab para moverse, Enter para botón por defecto).
- Se han definido atajos de teclado en el menú (Ctrl+A, Ctrl+B, etc.).
- El botón por defecto en la ventana de alta es “Guardar”, lo que permite confirmar con Enter después de rellenar.

Prueba en el proyecto:

Botón por defecto en VentanaAltaPiso:

text

```
// LISTENER DEL BOTÓN GUARDAR
// por defecto
getRootPane().setDefaultButton(bGuardar);
bGuardar.addActionListener(e -> {
    if (validarFormularios()) {
        JOptionPane.showMessageDialog(this, "Registro Guardado", "Registro Guardado",
            JOptionPane.INFORMATION_MESSAGE);
    }
});
}
```

Atajos en menú Ventana:

```
salir.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_F4, KeyEvent.CTRL_DOWN_MASK)); // Control + F4
altaPiso.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_A, KeyEvent.CTRL_DOWN_MASK)); // Control + A
bajaPiso.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_B, KeyEvent.CTRL_DOWN_MASK)); // Control + B
acercaDe.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_D, KeyEvent.CTRL_DOWN_MASK)); // Control + D
```

Comprensible

Definición:

La información y el uso de la interfaz deben ser comprensibles. Los usuarios deben poder entender el contenido y cómo interactuar con el sistema.

Ejemplos generales:

- Mensajes claros, sin tecnicismos.
- Etiquetas de campos sencillas y en el idioma del usuario.
- Validaciones con explicaciones.

Aplicación en SummerHeat:

- Los mensajes de error son claros y en castellano: “El DNI debe tener exactamente 8 dígitos y 1 letra MAYÚSCULA”.
- Las etiquetas de los campos son autoexplicativas (“Dirección”, “Provincia”, “Número de huéspedes”).
- Se usan tooltips para reforzar la comprensión en campos delicados (DNI, teléfono, extras niños).

Prueba en el proyecto:

Mensajes de validación:

```
String telefono = datosArrendado.txtTelefono.getText().trim();
if (telefono.isEmpty() || !telefono.matches("\\d{9}")) {
    JOptionPane.showMessageDialog(this, "El telefono debe tener exactamente 9 digitos.",
        "Error: Telefono invalido", JOptionPane.ERROR_MESSAGE);
    datosArrendado.txtTelefono.requestFocus();
    return false;
}

// Validar DatosInmueble
String direccion = datosInmueble.txtDireccion.getText().trim();
if (direccion.isEmpty()) {
    JOptionPane.showMessageDialog(this, "La direccion del inmueble es obligatoria.", "Error: Campo vacio",
        JOptionPane.ERROR_MESSAGE);
    datosInmueble.txtDireccion.requestFocus();
    return false;
}
```

Tooltips:

```
// Extras Niños
txtExtrasNinos = new JTextField(16);
txtExtrasNinos.setTooltipText("Extras para niños");
txtExtrasNinos.setPreferredSize(new Dimension(200, 21));
spEdadNinos = new JSpinner(new SpinnerNumberModel(0, 0, 10, 1));
```

Robusta

Definición:

El contenido debe ser lo suficientemente robusto como para ser interpretado de manera confiable por una amplia variedad de agentes de usuario, incluyendo tecnologías de asistencia (lectores de pantalla, diferentes sistemas).

Ejemplos generales:

- Uso de componentes estándar de interfaz en lugar de controles “raros”.
- Evitar controles hechos a medida sin etiquetado.
- Estructuras predecibles (formularios, etiquetas asociadas).

Aplicación en SummerHeat:

- Se usan componentes estándar de Swing: JLabel, JTextField, JSpinner, JComboBox, JCheckBox, JTabbedPane.

-
- La estructura es clara: cabecera, panel izquierdo, panel derecho, centro de imágenes, panel inferior de resumen.
 - No se usan dibujados personalizados que puedan confundir a tecnologías de asistencia.

Prueba en el proyecto:

Ejemplo de estructura estándar:

```
lNombre = new JLabel("Nombre: ");  
lApellido = new JLabel("Apellido: ");  
lDni = new JLabel("DNI: ");  
lTelefono = new JLabel("Telefono: ");  
  
txtNombre = new JTextField(16);  
txtApellido = new JTextField(16);
```

Pestañas con JTabbedPane:

```
// Añadir pestañas  
this.addTab("Arrendador", pArrendador);  
this.addTab("Inmueble", pInmueble);  
}
```

FIN - Esta parte no la considere tan tan importante hacerla a mano de principio a fin como el código, porque tenía ya un código de + 1500 líneas, y opte por hacerla con el chat que creo que para estos casos puntuales es donde se tiene que usar y no en hacer el código.